

Algoritmi

1. Fondamenti e Ordinamenti

1.1 Le Notazioni Asintotiche Minori (o piccolo e ω piccolo)

1.2 Il Limite Inferiore dell'Ordinamento

1.3 Ordinamenti Lineari: Oltre il muro di $\Omega(n \log n)$

1.A Radix Sort

1.B Bucket Sort

2. Strutture Dati Dinamiche e Alberi

2.1 Pile (Stack) e Code (Queue)

2.A Pila (Stack)

2.B Coda (Queue)

2.2 Liste Concatenate (Linked Lists)

I Costi delle Operazioni (FONDAMENTALE per i quiz):

3. Strutture Avanzate e Insiemi

3.1 Code di Priorità (L'Heap come struttura dinamica)

3.2 Alberi e Heap Binomiali

3.3 Insiemi Disgiunti (Union-Find)

4. Esplorazione Avanzata dei Grafi

4.1 I Superpoteri della DFS: Tempi e Archi

4.2 Ordinamento Topologico (Topological Sort)

4.3 Componenti Fortemente Connesse (SCC)

5. Cammini e Alberi di Copertura (Grafì Pesati)

5.1 Alberi di Copertura Minimi (MST - Minimum Spanning Tree)

5.A. Algoritmo di Kruskal

5.B. Algoritmo di Prim

5.2 Cammini Minimi a Sorgente Singola (Single-Source)

5.1.A Algoritmo di Dijkstra (Il Re dei pesi positivi)

5.1.B Algoritmo di Bellman-Ford (Il blindato)

6. I Paradigmi Algoritmici Supremi

6.1 Programmazione Dinamica (L'arte di ricordare)

6.2 Algoritmi Greedy (L'arte di accontentarsi subito)

6.3 Il Duello Finale (Il trabocchetto Supremo dell'esame)

7 Alberi Binari di Ricerca (BST)

7.1 La Proprietà Cardine dei BST

7.2 Le Tre Visite Fondamentali (Algoritmi Ricorsivi)

7.3 Operazioni di Base e Complessità (Il vero pericolo nei quiz)

7.4 La Cancellazione (Delete)

8 Alberi Rosso-Neri (RB-Trees)

8.1 Le 5 Proprietà dei Colori (Da sapere a memoria)

8.2 Le Rotazioni (Il meccanismo di aggiustamento)

9 Tabelle Hash

9.1 Il Concetto Base e il Problema Matematico

9.2 Metodo 1: Il Concatenamento (Chaining)

9.3 Metodo 2: L'Indirizzamento Aperto (Open Addressing)

9.4 La Trappola Mortale della Cancellazione

1. Fondamenti e Ordinamenti

Ricevuto. Entriamo nel vivo del Blocco 1 e andiamo a chiudere i buchi teorici che i professori amano usare per i trabocchetti nelle crocette.

Prendi l'iPad e segna solo i concetti chiave. Niente dimostrazioni chilometriche, andiamo dritti all'intuizione.

1.1 Le Notazioni Asintotiche Minori (*o* piccolo e ω piccolo)

Finora hai visto O (Big-O) e Ω (Big-Omega). Pensa a loro come ai classici simboli di disuguaglianza $\leq$ e $\geq$. Il Big-O dice "cresce meno o uguale", l'Omega dice "cresce di più o uguale".

Le notazioni "piccole" sono le versioni **strettamente maggiori o strettamente minori**:

- ***o* piccolo (o-little):** È l'equivalente del simbolo $<$. Se scrivi $f(n) = o(g(n))$, significa che $f(n)$ cresce *strettamente meno* di $g(n)$. Non la raggiungerà mai, nemmeno all'infinito.
 - *Esempio da crocetta:* $n^2 = o(n^3)$ è VERO. $n^2 = o(n^2)$ è FALSO (perché sono uguali, qui useresti il Big-O).
- **ω piccolo (omega-little):** È l'equivalente del simbolo $>$. Significa che la funzione cresce *strettamente di più*.
 - *Esempio da crocetta:* $n^3 = \omega(n^2)$ è VERO. $n^2 = \omega(n^2)$ è FALSO.

Il succo per gli appunti: $> O$ è $\leq$

o è $<$

Ω è $\geq$

ω è $>$

Θ è $=$

1.2 Il Limite Inferiore dell'Ordinamento

Questa è una domanda teorica garantita al 100%. Finora hai visto che Merge Sort e Heap Sort ci mettono $\Theta(n \log n)$. La domanda è: *è possibile inventare un algoritmo di ordinamento basato sui confronti che ci metta di meno, per esempio $O(n)$?*

La risposta è **NO, è matematicamente impossibile**. E la dimostrazione si fa tramite l'**Albero di Decisione**.

- **Cos'è:** Pensa a un albero dove ogni nodo interno è una domanda (es. "L'elemento A è maggiore di B?"), i rami sono le risposte (Sì/No), e le "foglie" finali sono l'array finalmente ordinato.
- **La logica:** Per un array di n elementi, ci sono $n!$ (n fattoriale) modi possibili in cui questi elementi possono essere mescolati all'inizio. Quindi, il nostro albero *deve* avere almeno $n!$ foglie per coprire tutte le combinazioni possibili.
- **La formula finale:** L'altezza di questo albero rappresenta il numero di confronti (il tempo). La matematica ci dice che un albero binario per avere $n!$ foglie deve avere un'altezza minima di $\Omega(n \log n)$.

Il succo per gli appunti: Nessun algoritmo che si basa sul *confrontare due elementi per volta* potrà mai scendere sotto il limite di $\Omega(n \log n)$. È il muro della fisica per questo tipo di algoritmi.

1.3 Ordinamenti Lineari: Oltre il muro di $\Omega(n \log n)$

Ma allora come fa il Counting Sort a ordinare in $O(n)$, violando il teorema che abbiamo appena visto? Semplice: *non usa i confronti*. Usa i numeri stessi come indici di un array.

A completare la famiglia del Counting Sort ci sono altri due algoritmi che ignorano i confronti e battono il tempo lineare:

1.A Radix Sort

Come ordini numeri enormi (es. numeri di telefono o date di nascita) senza far esplodere la memoria del Counting Sort?

- **Il meccanismo:** Ordina i numeri cifra per cifra, partendo dalla cifra meno significativa (quella più a destra, le unità) fino ad arrivare a quella più significativa (LSD - Least Significant Digit).
- **Il segreto:** Per ordinare la singola colonna di cifre, il Radix Sort chiama come "aiutante" un altro algoritmo, di solito proprio il Counting Sort.
- **Perché funziona:** Funziona *solo* se l'algoritmo aiutante è **stabile** (ricordi la tua domanda sull'ordine mantenuto?). Se il Counting Sort non fosse stabile, quando passi a ordinare le decine scombineresti l'ordine perfetto che avevi creato sulle unità.
- **Complessità:** $\Theta(d(n + k))$ dove d è il numero di cifre.

1.B Bucket Sort

Immagina di dover ordinare i voti in centesimi di mille studenti universitari.

- **Il meccanismo:** Crea una serie di "secchi" (buckets), ad esempio un secchio per i voti da 0 a 10, uno per 11-20, ecc. Butta ogni numero nel secchio corrispondente.
- **La chiusura:** Ordina ogni singolo secchio internamente (di solito usando l'Insertion Sort, che è fulmineo su array piccolissimi). Poi concatena i secchi e hai finito.
- **Complessità:** Se i numeri sono distribuiti in modo abbastanza uniforme (cioè non finiscono tutti nello stesso secchio), il tempo medio è $O(n)$.

2. Strutture Dati Dinamiche e Alberi

2.1 Pile (Stack) e Code (Queue)

Sono strutture lineari in cui l'accesso agli elementi è strettamente regolato da una politica di priorità. Nelle crocette si concentrano sempre su due concetti: la politica di accesso e il costo delle operazioni.

2.A Pila (Stack)

- **Politica: LIFO** (*Last In, First Out*). L'ultimo elemento che entra è il primo a uscire. Pensa a una pila di piatti: puoi aggiungere o togliere solo da sopra.
- **Operazioni fondamentali:** * `Push(S, x)` : inserisce l'elemento x in cima alla pila.
 - `Pop(S)` : rimuove e restituisce l'elemento in cima.
- **Costo computazionale:** $O(1)$ per entrambe le operazioni. Non importa quanti elementi ci siano nella pila, l'accesso alla cima è immediato.

2.B Coda (Queue)

- **Politica: FIFO** (*First In, First Out*). Il primo che entra è il primo a uscire. Pensa alla fila alle casse del supermercato.
- **Operazioni fondamentali:**
 - `Enqueue(Q, x)` : inserisce l'elemento x in coda (alla fine).
 - `Dequeue(Q)` : rimuove e restituisce l'elemento in testa (all'inizio).
- **Costo computazionale:** $O(1)$ per entrambe le operazioni.

Trabocchetto da crocetta: Se implementi una coda con un array circolare, le operazioni mantengono costo $O(1)$. Se l'implementazione è inefficiente e richiede lo slittamento degli elementi, il costo può salire a $O(n)$, ma nello standard teorico le risposte corrette sono sempre $O(1)$.

2.2 Liste Concatenate (Linked Lists)

A differenza degli array, gli elementi di una lista non sono contigui in memoria. Ogni elemento (nodo) contiene il dato e un puntatore al nodo successivo.

- **Lista Semplicemente Concatenata:** Ogni nodo punta solo al successivo (`next`). L'ultimo nodo punta a `NIL`.
- **Lista Doppia Concatenata:** Ogni nodo ha due puntatori: uno al successivo (`next`) e uno al precedente (`prev`). Ti permette di navigare la lista in entrambe le direzioni.

I Costi delle Operazioni (FONDAMENTALE per i quiz):

Molti studenti confondono i costi delle liste con quelli degli array. Memorizza questa tabella:

Operazione	Array	Lista Concatenata	Note per la Lista
Ricerca (Search)	$O(n)$ (o $O(\log n)$ se ordinato)	$O(n)$	Devi scorrere i nodi uno a uno partendo dalla testa.
Inserimento (Insert)	$O(n)$ (se devi spostare elementi)	$O(1)$	Basta aggiornare i puntatori del nuovo nodo in testa.
Cancellazione (Delete)	$O(n)$	$O(1) \circ O(n)$	<i>Attenzione:</i> Cancellare un nodo dato il suo puntatore in una lista doppia costa $O(1)$. Se invece devi prima <i>cercare</i> l'elemento da cancellare, il costo è dominato dalla ricerca e diventa $O(n)$.

3. Strutture Avanzate e Insiemi

3.1 Code di Priorità (L'Heap come struttura dinamica)

Nel Blocco 2 hai visto l'Heap come un trucco per ordinare un array (l'Heap Sort). Ma la sua vera natura è un'altra: è una **Coda di Priorità**. Pensa al pronto soccorso: non passa il primo che arriva (come nella coda classica), ma passa chi ha il codice rosso (la priorità più alta).

Un Max-Heap è perfetto per questo perché tiene sempre l'elemento più grande (o più importante) in cima (alla radice).

Le 4 operazioni fondamentali (e i loro costi da sapere a memoria):

- **Maximum(S)** - $O(1)$: Chi è il più grave in sala d'attesa? Guardo la radice e lo so all'istante.
- **Extract-Max(S)** - $O(\log n)$: Faccio entrare il paziente più grave. Rimuovo la radice, prendo l'ultimo elemento dell'array, lo metto al posto della radice e chiamo **Max-Heapify** per farlo "sprofondare" nella posizione corretta.

- **Insert(S, x)** - $O(\log n)$: Arriva un nuovo paziente. Lo metto in fondo all'array e lo faccio "galleggiare" verso l'alto finché non trova il suo livello di gravità corretto.
- **Increase-Key(S, x, k)** - $O(\log n)$: Un paziente in sala d'attesa peggiora improvvisamente. Aumento il suo valore e lo faccio "galleggiare" verso l'alto.

Trabocchetto mortale da crocetta: Se ti chiedono quanto costa costruire un Heap partendo da un array disordinato (la funzione **Build-Max-Heap**), la risposta istintiva di molti è $O(n \log n)$ perché si fanno n inserimenti. **È FALSO**. La matematica dimostra che **Build-Max-Heap** costa $O(n)$. Segnalo in rosso fuoco.

3.2 Alberi e Heap Binomiali

Se l'Heap normale è così bello, perché studiare gli Heap Binomiali?

Perché l'Heap normale ha un difetto gravissimo: **unire due Heap diversi costa $O(n)$** . Se devi fondere due ospedali, è un disastro. L'Heap Binomiale risolve questo problema permettendo una fusione in $O(\log n)$.

- **La struttura visiva:** Non è un solo albero, ma una "foresta" di alberi chiamati B_k .
 - B_0 è un singolo nodo.
 - B_1 sono due B_0 uniti.
 - In generale, per creare un albero B_k , prendi due alberi B_{k-1} e attacchi la radice di uno sotto la radice dell'altro.
- **Proprietà matematica (Croce e delizia dei quiz):** Un albero B_k ha esattamente 2^k **nodi** e un'altezza pari a k .
- **Il trucco della fusione (Merge):** Unire due Heap Binomiali funziona *esattamente come l'addizione binaria*. Se hai due alberi dello stesso grado (es. due B_2), li unisci e formano un "riporto" (un B_3), proprio come $1 + 1 = 10$ in binario.

Il succo per gli appunti: Heap Normale = Ottimo per inserire/estrarre, pessimo per unire. Heap Binomiale = Ottimo per unire ($O(\log n)$).

3.3 Insiemi Disgiunti (Union-Find)

Questa è la struttura dati più "magica" del corso. Serve a gestire gruppi di elementi separati e a rispondere velocemente a una domanda: *"Questi due elementi fanno parte dello stesso gruppo?"* (Ti servirà come il pane per l'Algoritmo di Kruskal sui grafi).

Le 3 operazioni fondamentali:

- **Make-Set(x)** : Crea un nuovo gruppo con solo x dentro.
- **Find-Set(x)** : Restituisce il "rappresentante" del gruppo in cui si trova x . Se **Find-Set(A) == Find-Set(B)** , allora A e B sono nello stesso gruppo.
- **Union(x, y)** : Fonde il gruppo di x con il gruppo di y .

Le 2 Euristiche (Da sapere PERFETTAMENTE):

Per rendere queste operazioni fulminee, si usano due trucchi combinati:

1. **Unione per Rango (Union by Rank)**: Quando unisci due alberi, attacchi *sempre* la radice dell'albero più basso sotto la radice dell'albero più alto. Questo evita che l'albero diventi uno spaghetti lunghissimo.
2. **Compressione dei Cammini (Path Compression)**: Quando fai una **Find-Set** e percorri l'albero dal basso verso la radice, prendi tutti i nodi che hai toccato e li stacchi, riattaccandoli *direttamente* alla radice. La volta successiva la ricerca sarà istantanea.

Il miracolo matematico: Se usi entrambe queste euristiche, il costo delle operazioni diventa $O(\alpha(n))$, dove α è l'inversa della funzione di Ackermann. Ai fini pratici (e delle crocette), $\alpha(n)$ cresce così lentamente che **il costo è considerato quasi costante: praticamente $O(1)$.**

4. Esplorazione Avanzata dei Grafi

4.1 I Superpoteri della DFS: Tempi e Archi

Quando fai una DFS, non ti limiti a "visitare" i nodi, ma tieni un cronometro globale. Ogni nodo v riceve due "timbrì temporali" (timestamps):

- **Tempo di scoperta ($d[v]$):** Il momento esatto in cui il nodo diventa grigio (scoperto per la prima volta).
- **Tempo di fine visita ($f[v]$):** Il momento esatto in cui il nodo diventa nero (hai finito di esplorare tutti i suoi vicini e chiudi la chiamata ricorsiva).

Da questi due numerini derivano le due regole d'oro per le crocette:

A. Il Teorema delle Parentesi:

Se rappresenti la scoperta di un nodo con una parentesi aperta $(v$ e la sua fine con una parentesi chiusa $)v$, la storia di tutta la DFS forma un'espressione matematica perfettamente annidata.

- **Regola da test:** L'intervallo $[d[v], f[v]]$ di un nodo o è **completamente contenuto** in quello di un altro (è suo discendente), o è **completamente disgiunto** (sono in rami diversi). Non si accavallano mai a metà.

B. La Classificazione degli Archi:

Durante la DFS, ogni arco del grafo viene etichettato. Devi saperli riconoscere:

1. **Archi dell'Albero:** Sono gli archi "normali" che scoprono nuovi nodi bianchi.
2. **Archi all'Indietro (Back Edges):** Puntano da un nodo a un suo antenato (un nodo grigio, ancora aperto). **Domanda da esame fissa:** *Come si rileva un ciclo in un grafo?* Risposta: *Se e solo se la DFS trova almeno un arco all'indietro.*
3. **Archi in Avanti (Forward Edges):** Puntano da un nodo a un suo discendente (già nero) che non è un figlio diretto.
4. **Archi di Attraversamento (Cross Edges):** Tutti gli altri. Puntano a nodi già neri che stanno su rami completamente diversi.

4.2 Ordinamento Topologico (Topological Sort)

Hai un grafo che rappresenta le propedeuticità degli esami (es. devi dare Analisi 1 prima di Analisi 2). In che ordine devi dare gli esami? L'ordinamento topologico li mette in fila da sinistra a destra, in modo che tutte le frecce puntino solo verso destra.

- **Il Requisito Fondamentale (Memorizzalo):** Funziona **SOLO** sui **DAG** (Directed Acyclic Graphs, Grafo Diretto Aciclico). Se c'è un ciclo (es. A richiede B e B richiede A), è impossibile ordinarli.

- **L'Algoritmo Magico (è banalissimo):** 1. Lancia una normale DFS su tutto il grafo.
2. Ogni volta che un nodo *finisce* (diventa nero, tempo $f[v]$), inseriscilo in testa a una lista concatenata.
3. Fine. La lista è il tuo grafo ordinato.
- **Complessità:** Visto che è letteralmente una DFS, il costo è $\Theta(V + E)$ (Vertici + Archi).

4.3 Componenti Fortemente Connesse (SCC)

Una SCC (Strongly Connected Component) è un gruppo esclusivo di nodi "vip" all'interno di un grafo diretto: partendo da un qualsiasi nodo del gruppo, puoi raggiungere *tutti* gli altri nodi di quel gruppo.

- **L'Algoritmo (Spesso chiamato algoritmo di Kosaraju):**

Questo è un pattern da 3 step che amano chiedere per testare se sai combinare gli algoritmi:

1. Lancia una DFS sul grafo originale G per calcolare tutti i tempi di fine visita $f[u]$.
 2. Calcola il **Grafo Trasposto** G^T (cioè prendi il grafo originale e inverti la direzione di tutte le frecce/archi). Crearlo costa $\Theta(V + E)$.
 3. Lancia una seconda DFS sul grafo trasposto G^T . Ma attenzione: nel ciclo principale (quello che lancia le visite sui nodi bianchi), processa i nodi **in ordine decrescente rispetto ai tempi di fine** $f[u]$ che avevi calcolato allo step 1.
 4. *Risultato:* Ogni singolo albero che si forma in questa seconda DFS è esattamente una Componente Fortemente Connessa.
- **Complessità totale:** $\Theta(V + E)$. Sono solo due DFS e un'inversione di archi.

5. Cammini e Alberi di Copertura (Grafì Pesati)

5.1 Alberi di Copertura Minimi (MST - Minimum Spanning Tree)

Il problema: Hai un grafo che rappresenta delle città. Vuoi stendere dei cavi in fibra ottica per collegarle tutte, ma vuoi spendere il minimo indispensabile. La soluzione ottimale non avrà mai cicli (sarebbero cavi sprecati), quindi formerà sempre un albero.

Per risolvere questo problema usiamo due algoritmi famosissimi. **Entrambi usano un approccio Greedy** (fanno la scelta che sembra migliore sul momento).

5.A. Algoritmo di Kruskal

- **La logica:** Guarda tutti gli archi del grafo, li mette in fila dal più economico al più costoso, e inizia a prenderli uno a uno.
- **Il problema:** Prendendo archi a caso per la mappa, c'è il rischio di creare un ciclo.
- **L'arma segreta:** Usa la struttura dati **Union-Find** (che abbiamo visto prima). Prima di aggiungere un arco, chiede alla Union-Find: *"Questi due nodi sono già collegati?"*. Se sì, scarta l'arco. Se no, li unisce.
- **Complessità:** Visto che deve ordinare tutti gli archi all'inizio, il costo è dominato dall'ordinamento: $O(E \log E)$ oppure $O(E \log V)$ (matematicamente sono equivalenti nei grafi).

5.B. Algoritmo di Prim

- **La logica:** Invece di guardare gli archi sparsi, Prim parte da un nodo casuale e fa crescere un unico albero centrale, come una macchia d'olio. Ad ogni passo espande la macchia aggiungendo l'arco più economico che si affaccia verso l'esterno.
- **L'arma segreta:** Usa una **Coda di Priorità (Min-Heap)**. Ci butta dentro tutti i nodi e ad ogni passo estrae quello "più vicino" all'albero in costruzione.
- **Complessità:** Dipende da come implementi la coda. Con un Min-Heap normale costa $O(E \log V)$.

Trabocchetto da crocetta: Se un grafo ha tutti archi con pesi *diversi*, ci sarà un solo identico MST alla fine, e sia Prim che Kruskal costruiranno **esattamente lo stesso albero**, anche se in ordine diverso.

5.2 Cammini Minimi a Sorgente Singola (Single-Source)

Il problema: È letteralmente Google Maps. Parti da un nodo specifico (es. casa tua) e vuoi trovare il percorso più breve per arrivare a *tutti* gli altri nodi del grafo.

Anche qui abbiamo due fuoriclasse, e la scelta di chi usare dipende da un solo fattore vitale: **i pesi negativi**.

5.1.A Algoritmo di Dijkstra (Il Re dei pesi positivi)

- **La logica:** È un algoritmo **Greedy**. Crea una "bolla" di nodi di cui sa la distanza definitiva. Ad ogni passo esplora i vicini, aggiorna le distanze provvisorie e fa entrare nella bolla il nodo più vicino in assoluto.
- **L'arma segreta:** Usa una **Coda di Priorità (Min-Heap)** per trovare istantaneamente il nodo più vicino da esplorare.
- **Complessità:** Con un Min-Heap binario costa $O((V + E) \log V)$.
- **Il tallone d'Achille (DOMANDA FISSA ALL'ESAME):** Dijkstra è cieco ai **pesi negativi**. Se nel grafo c'è un arco con peso negativo (es. una macchina del tempo o un guadagno invece che un costo), Dijkstra sballa completamente i calcoli perché la sua logica Greedy dà per scontato che aggiungere archi possa solo far aumentare la distanza, mai diminuirla.

5.1.B Algoritmo di Bellman-Ford (Il blindato)

- **La logica:** Non usa code intelligenti, usa la forza bruta intelligente (è un assaggio di Programmazione Dinamica). Prende *tutti* gli archi del grafo e li "rilassa" (cioè controlla se offrono una scorciatoia). Ripete questa operazione di blocco per un numero di volte pari a $V - 1$ (Vertici meno uno).
- **I superpoteri:** Bellman-Ford se ne frega dei pesi negativi, li calcola perfettamente. Ma fa di più: **rileva i cicli di peso negativo**. Se un grafo ha un ciclo in cui la somma dei pesi è sotto zero, puoi girarci all'infinito abbassando la distanza a $-\infty$. Bellman-Ford se ne accorge e restituisce **FALSE** (distanza impossibile da calcolare).
- **Complessità:** Visto che processa tutto per tutti, è lento. Il suo costo è $O(V \cdot E)$.

Il succo per gli appunti: > Tutti i pesi positivi? → Vai di **Dijkstra** (veloce).

Pesi negativi o sospetto di cicli negativi? → Vai di **Bellman-Ford** (lento ma sicuro).

6. I Paradigmi Algoritmici Supremi

6.1 Programmazione Dinamica (L'arte di ricordare)

Il problema: Immagina di calcolare la sequenza di Fibonacci in modo ricorsivo. Per calcolare $F(5)$, devi calcolare $F(4)$ e $F(3)$. Ma per calcolare $F(4)$, devi ricalcolare *di nuovo* $F(3)$. È un disastro: l'algoritmo ricalcola le stesse cose milioni di volte, facendo esplodere il tempo a livelli esponenziali.

La soluzione (Dinamica): Usa la memoria. Ogni volta che risolvi un sotto-problema, salvi il risultato in una tabella. Se il programma ti richiede di nuovo quel calcolo, non lo rifai: vai a leggere la tabella. Un problema esponenziale diventa magicamente lineare o quadratico.

I due requisiti di base (DOMANDA FISSA DA CROCETTA):

Per poter usare la Programmazione Dinamica, il problema *deve* avere queste due proprietà:

1. **Sottostruttura Ottima:** La soluzione ottima del problema grande contiene al suo interno le soluzioni ottime dei problemi più piccoli.
2. **Sottoproblemi Sovrapponibili:** L'algoritmo ricorsivo ingenuo continua a richiamare gli stessi identici problemi più e più volte (come in Fibonacci).

I due approcci (Altra potenziale crocetta):

- **Top-Down (Memoizzazione):** Scrivi l'algoritmo in modo ricorsivo (parti dal grande e vai verso il piccolo), ma prima di fare un calcolo controlla se è già salvato in un array di "appunti" (il *memo*).
- **Bottom-Up (Tabulazione):** Abbandona la ricorsione. Usi dei semplici cicli `for` per riempire una matrice partendo dai casi base (i problemi più piccoli) e salendo piano piano fino alla soluzione finale.

6.2 Algoritmi Greedy (L'arte di accontentarsi subito)

Il problema: La Programmazione Dinamica è infallibile, ma a volte riempire enormi matrici è troppo lento o consuma troppa memoria.

La soluzione (Greedy): Invece di valutare tutte le opzioni possibili come fa la Dinamica, un algoritmo Greedy (ingordo) fa la scelta che gli sembra **migliore in quel preciso momento**, senza mai guardarsi indietro e senza mai pentirsi.

Hai presente Dijkstra e Kruskal? Sono algoritmi Greedy. Dijkstra prende sempre il nodo più vicino in quel momento, Kruskal prende sempre l'arco più economico.

I due requisiti di base:

1. **Sottostruttura Ottima:** (In comune con la Dinamica).
2. **Proprietà della Scelta Ingorda (Greedy Choice Property):** È la differenza fondamentale. Significa che puoi arrivare alla soluzione globale ottima semplicemente facendo scelte localmente ottime passo dopo passo, senza dover esplorare le alternative.

6.3 Il Duello Finale (Il trabocchetto Supremo dell'esame)

C'è un esempio classico che i professori usano *sempre* per testare se hai capito la differenza tra Dinamica e Greedy: **Il Problema dello Zaino (Knapsack Problem)**.

Immagina di essere un ladro in un negozio, hai uno zaino che tiene massimo 50 kg e devi scegliere quali oggetti rubare per massimizzare il valore.

- **Lo Zaino Frazionario (Risolubile con Greedy):** Se puoi tagliare gli oggetti (es. polvere d'oro, farina), puoi usare un approccio Greedy. Calcoli il valore al chilo di ogni oggetto, prendi tutto quello che puoi di quello che vale di più, poi passi al secondo e così via. Finché lo zaino è pieno. Rapido e perfetto.
- **Lo Zaino 0-1 (Risolubile SOLO con Dinamica):** Gli oggetti non si possono tagliare (es. un televisore, un lingotto intero). O lo prendi (1) o lo lasci (0). In questo caso l'approccio Greedy **fallisce miseramente** (potresti riempire lo zaino con un oggetto che vale tanto al chilo ma che lascia un sacco di spazio vuoto inutilizzabile). L'unica soluzione che garantisce il risultato perfetto è la Programmazione Dinamica (si costruisce una matrice in cui si valuta "peso vs beneficio" per ogni singolo oggetto).

Il succo per gli appunti: Se vedi "Zaino Frazionario" → **Greedy**. Se vedi "Zaino 0-1" → **Programmazione Dinamica**.

7 Alberi Binari di Ricerca (BST)

Gli Alberi Binari di Ricerca (BST - Binary Search Trees) rappresentano una delle strutture dati più importanti dell'intero corso. Risolvono il grande limite delle liste: permettono di cercare, inserire e cancellare elementi in tempo logaritmico, mantenendo i dati ordinati.

Prepara l'iPad e segna queste regole fondamentali, strutturate esattamente per smascherare i trabocchetti delle crocette.

7.1 La Proprietà Cardine dei BST

In un albero binario di ricerca, la disposizione dei nodi segue una regola geometrica e matematica rigida:

Sia x un nodo dell'albero.

- Se y è un nodo nel **sottoalbero sinistro** di x , allora $y.chiave \leq x.chiave$.
- Se y è un nodo nel **sottoalbero destro** di x , allora $y.chiave \geq x.chiave$.

7.2 Le Tre Visite Fondamentali (Algoritmi Ricorsivi)

Per scorrere i nodi di un albero esistono tre strategie ricorsive, differenziate dall'ordine in cui viene elaborata la radice rispetto ai figli:

- **In-Order (Asimmetrica):** Visita il sottoalbero sinistro, elabora la radice, visita il sottoalbero destro.
 - **TRABOCCHETTO DA ESAME GARANTITO:** La visita *In-Order* di un BST restituisce sempre le chiavi in ordine **strettamente crescente**. Se una crocetta ti chiede come ordinare un BST, la risposta è la visita In-Order.
- **Pre-Order (Anticipata):** Elabora la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. (Utile per creare una copia esatta dell'albero).
- **Post-Order (Posticipata):** Visita il sottoalbero sinistro, visita il sottoalbero destro, elabora la radice. (Utile quando devi eliminare l'albero, perché distrugge i figli prima di eliminare il padre).

7.3 Operazioni di Base e Complessità (Il vero pericolo nei quiz)

Tutte le operazioni fondamentali (`Search` , `Insert` , `Delete` , `Minimum` , `Maximum` , `Successor` , `Predecessor`) hanno una complessità strettamente legata all'**altezza** h dell'albero.

Il costo di qualsiasi operazione di ricerca, inserimento o cancellazione è $O(h)$.

Il grande inganno delle crocette sulle prestazioni:

- **Caso Ottimo / Bilanciato:** Se l'albero è ben bilanciato, l'altezza è minima: $h = \log n$. In questo caso, le operazioni costano $O(\log n)$.
- **Caso Peggior / Degenere:** Se inserisci i numeri già ordinati (es. 1, 2, 3, 4, 5), l'albero diventa una linea retta (uno "stecco" o una lista concatenata). In questo caso, l'altezza è massima: $h = n$. Di conseguenza, le operazioni crollano a una complessità di $O(n)$.

Regola d'oro per l'appunto: Non rispondere mai $O(\log n)$ in modo assoluto se la domanda non specifica che l'albero è bilanciato. Senza vincoli, la risposta corretta per il caso peggiore è sempre $O(n)$.

7.4 La Cancellazione (`Delete`)

Mentre la ricerca e l'inserimento sono intuitivi (basta scendere a sinistra o a destra confrontando le chiavi), la cancellazione di un nodo richiede la gestione di 3 casi distinti:

1. **Il nodo è una foglia:** Non ha figli. Si cancella semplicemente staccandolo dal padre ($O(1)$).
2. **Il nodo ha un solo figlio:** Si elimina il nodo e si collegano direttamente il padre e il figlio del nodo eliminato (come se fosse una lista, $O(1)$).
3. **Il nodo ha due figli:** Non possiamo distruggere la struttura. Si cerca il **successore** del nodo (il nodo più piccolo nel sottoalbero destro), si copia il suo valore nel nodo da eliminare e poi si cancella fisicamente il successore (che ricadrà sempre nel caso 1 o nel caso 2).

8 Alberi Rosso-Neri (RB-Trees)

Eccoci all'ultima, fondamentale struttura per completare il quadro degli alberi di ricerca: gli **Alberi Rosso-Neri (RB-Trees)**.

Come abbiamo visto, i normali BST sono perfetti in teoria, ma nella pratica rischiano di degenerare in semplici liste (casi peggiori da $O(n)$) se i dati vengono inseriti già ordinati. Gli alberi Rosso-Neri sono la soluzione elegante e geniale a questo problema: si **auto-bilanciano**, garantendo *sempre* un'altezza massima proporzionale a $\log n$ e, di conseguenza, operazioni di costo sempre pari a $O(\log n)$.

8.1 Le 5 Proprietà dei Colori (Da sapere a memoria)

Per essere un vero "Rosso-Nero", un albero binario di ricerca deve rispettare 5 regole ferree. Se anche una sola viene violata durante un inserimento o una cancellazione, l'albero deve aggiustarsi (vedremo come tra un attimo).

1. **Proprietà del Colore:** Ogni nodo dell'albero è o **Rosso** o **Nero**.
2. **Proprietà della Radice:** La radice dell'albero è *sempre* **Nera**.
3. **Proprietà delle Foglie:** Tutte le foglie (cioè i nodi vuoti **NIL** che chiudono l'albero in basso) sono considerate **Nere**.
4. **Proprietà dei Rossi (La regola anti-filotto):** Se un nodo è **Rosso**, *entrambi* i suoi figli devono essere **Neri**. (Corollario: non possono esistere due nodi rossi consecutivi lungo un percorso).
5. **Proprietà del Cammino Nero (La regola del bilanciamento):** Per *qualsiasi* nodo x , tutti i cammini semplici che partono da x e scendono fino a una foglia **NIL** devono contenere esattamente lo stesso numero di nodi **Neri**. (Questo numero è chiamato "altezza nera" del nodo x).

Il succo per le crocette: È la regola 4 (niente rossi consecutivi) combinata con la regola 5 (stesso numero di neri su ogni cammino) che costringe l'albero a rimanere basso e tozzo, impedendogli di diventare un lungo stecco degenerato.

8.2 Le Rotazioni (Il meccanismo di aggiustamento)

Quando inserisci un nuovo nodo, di base lo colori sempre di **Rosso** (per non violare subito l'altezza nera, la regola 5). Ma così facendo, rischi di creare un nodo Rosso figlio di un altro nodo Rosso, violando la regola 4.

Come fa l'albero a "curarsi"? Oltre a poter semplicemente ricolorare i nodi (cambiare i rossi in neri e viceversa), usa un'operazione chirurgica e fulminea

chiamata **Rotazione**.

- **Cos'è:** Una rotazione è uno scambio di posizioni locali tra un padre e un figlio, che mantiene intatta la proprietà di ordinamento del BST, ma sposta il "peso" (l'altezza) da un lato all'altro.
- **Rotazione a Sinistra (`Left-Rotate(T, x)`):** Prendi il figlio destro di x e lo tiri su facendolo diventare il nuovo padre. Il vecchio padre x "scivola" giù e diventa il figlio sinistro del nuovo padre.
- **Rotazione a Destra (`Right-Rotate(T, y)`):** È l'esatto opposto. Prendi il figlio sinistro di y e lo tiri su. Il vecchio padre y scivola e diventa il figlio destro.
- **Costo:** La bellezza delle rotazioni è che coinvolgono solo il cambio di 3 o 4 puntatori in memoria. Il loro costo è puramente $O(1)$, indipendentemente dalla grandezza dell'albero.

Trabocchetto da Esame: Nelle crocette spesso chiedono il costo massimo di inserimento in un albero Rosso-Nero. La risposta è $O(\log n)$ (il costo per scendere lungo l'albero a trovare il posto), ma le rotazioni necessarie per ripristinare i colori costano solo $O(1)$ (nella peggiore delle ipotesi se ne fanno al massimo due).

9 Tabelle Hash

9.1 Il Concetto Base e il Problema Matematico

Immagina di avere una tabella (un array) di dimensione m (es. 10 posti) e un universo di chiavi possibili U enorme (es. tutti i codici fiscali italiani).

- **La Funzione Hash $h(k)$:** È un tritatutto matematico. Prende una chiave k e spara fuori un numero da 0 a $m - 1$, dicendoti in quale cella dell'array devi metterla. (Il metodo più classico è la divisione: $h(k) = k \bmod m$).
- **La Collisione:** Per il *Principio dei Cassetti*, se hai 11 piccioni e 10 cassetti, almeno un cassetto avrà due piccioni. Visto che U è molto più grande di m , prima o poi la funzione hash manderà **due chiavi diverse nello stesso identico slot**.

Tutte le crocette sulle Tabelle Hash vertono su **come si risolve questa collisione**.
Ci sono due scuole di pensiero:

9.2 Metodo 1: Il Concatenamento (Chaining)

È il metodo più semplice e pacifico. Se due elementi finiscono nello stesso slot, non si rubano il posto: lo slot diventa semplicemente la "testa" di una **lista concatenata**.

- **Fattore di Carico (α):** È il numero medio di elementi per ogni slot. Si calcola come $\alpha = n/m$ (dove n sono gli elementi totali inseriti e m i posti nella tabella).
- **Complessità (Domanda fissa):** L'inserimento costa sempre $O(1)$ (lo metti in testa alla lista). La ricerca, nel caso medio, costa $\Theta(1 + \alpha)$.

Il succo per le crocette: Finché il numero di elementi n è proporzionale ai posti m (quindi α è una piccola costante), la ricerca rimane fulminea a $O(1)$. Se metti un milione di elementi in una tabella da 10 posti, α esplode e la ricerca diventa lenta come una lista $O(n)$.

9.3 Metodo 2: L'Indirizzamento Aperto (Open Addressing)

Qui vietiamo le liste. Ogni elemento **deve** vivere dentro una cella della tabella. Nessun puntatore esterno, memoria usata al 100%.

Ma se il posto è già occupato? Si va a tentativi (ispezioni), cercando il primo posto libero.

Ci sono tre modi per calcolare i tentativi successivi:

1. **Ispezione Lineare:** Il posto è occupato? Guardo quello subito dopo ($+1$), poi il successivo, ecc.
 - *Trabocchetto (Il suo difetto fatale):* Soffre di **Agglomerazione Primaria** (Primary Clustering). Si formano dei "tappi" di elementi vicini, e ogni nuovo scontro allunga il tappo, rallentando drasticamente le ricerche future.
2. **Ispezione Quadratica:** Faccio salti via via più lunghi ($+1^2, +2^2, +3^2$). Evita i tappi compatti, ma soffre di agglomerazione *secondaria* (se due chiavi partono dallo stesso slot, faranno esattamente gli stessi salti).

3. **Doppio Hashing (Il Perfetto):** Uso *due* funzioni hash diverse. La prima mi dà la posizione di partenza, la seconda mi dice quanto lungo deve essere il passo dei miei salti. Questo disperde i dati in modo pseudo-casuale e perfetto.

9.4 La Trappola Mortale della Cancellazione

Questa è forse **la crocetta più gettonata in assoluto** sulle Tabelle Hash.

Domanda: Come si cancella un elemento in una tabella hash con Indirizzamento Aperto?

- *Risposta Istintiva (Sbagliata):* "Lo tolgo e rimetto la cella vuota (NIL)".
- *Risposta Corretta:* "Non puoi metterlo a NIL. Devi sostituirlo con un valore speciale chiamato **DELETED**".

Perché?

Immagina di inserire A , poi inserisci B che ha lo stesso hash di A . Con l'ispezione lineare, metti B nello slot subito dopo A .

Se un giorno cancelli A e metti la cella a **NIL**, quando cercherai B la funzione andrà nel primo slot, troverà **NIL** e penserà: "Ah, la cella è vuota, quindi B non è mai stato inserito!".

Mettendo il marcatore **DELETED**, la ricerca capisce che deve scavalcare quel cadavere e continuare a cercare.